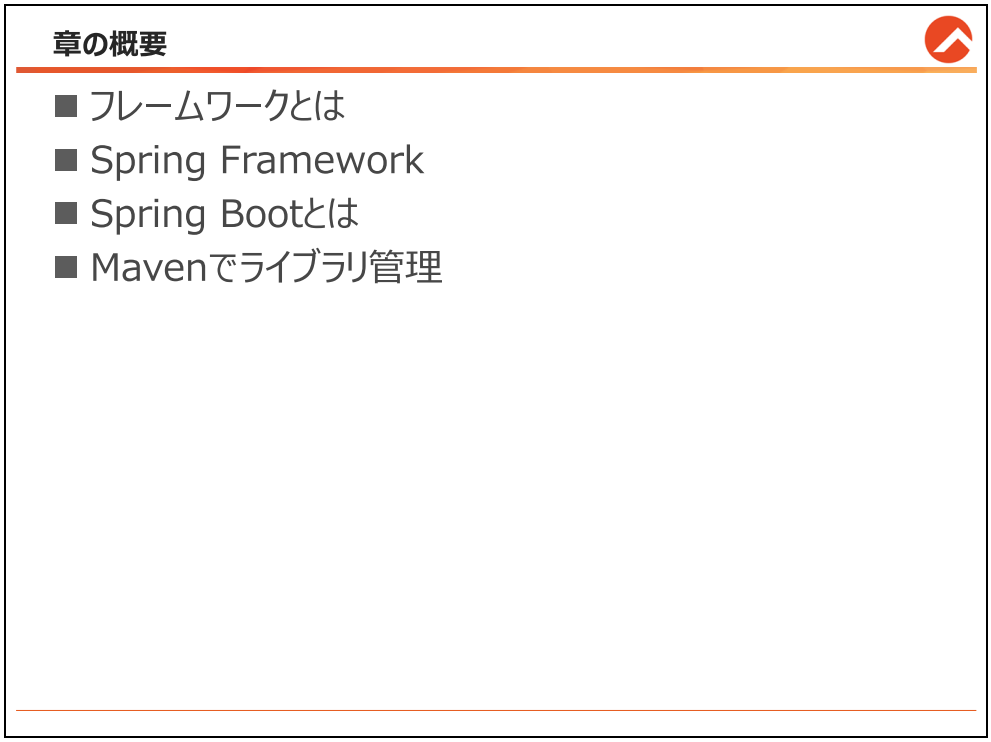
1

# 第1章 Spring Boot と開発環境の概要

## 章の目標

- フレームワークとSpring Framework について理解する
- Spring Boot の基本概念と特徴を把握する
- Maven を用いてライブラリのインストールができる

# 章の概要

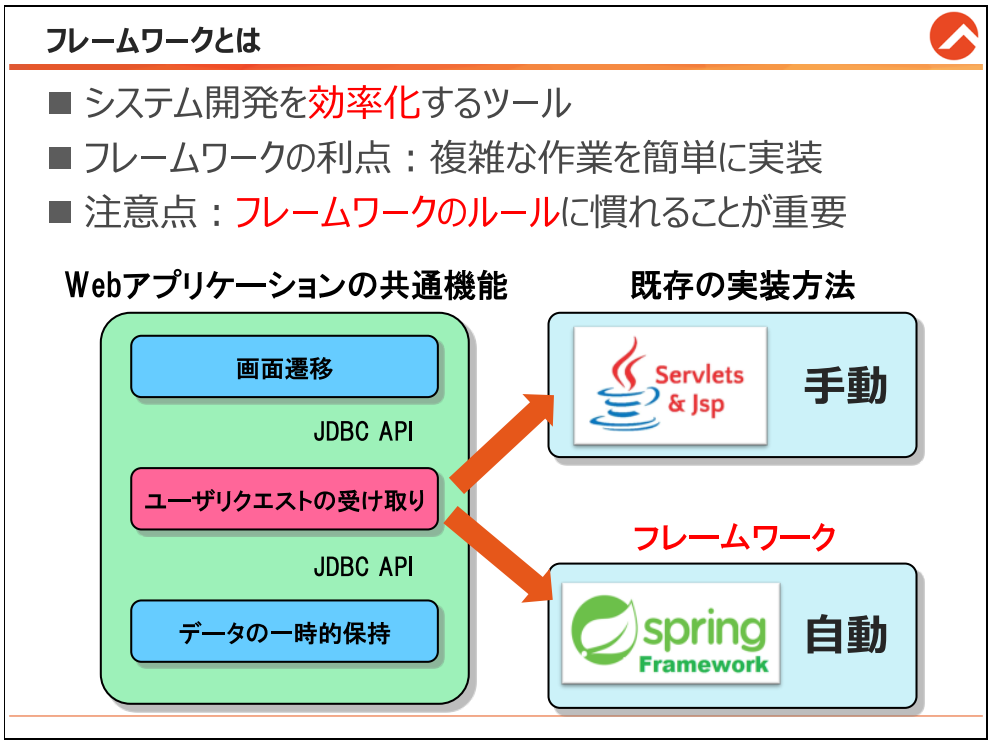



この章では、Java でのWeb 開発における、フレームワークの概念とそのフレームワークの中でもよく活用されるSpring Boot の基本概念について学習します。

続いて、Spring Boot のプロジェクトの作成方法について学習します。Spring Boot は、プロジェクトを作成する時、しっかり設定をすることで、元々はややこしい環境設定を簡単にできるようになるので、しっかり習得する必要があります。

そして、フレームワークの環境設定で欠かせないビルドツールである、Maven についてもご紹介します。Maven を学ぶことにより、ライブラリの導入と、ビルドを効率化することができます。

## 1.1. フレームワークとは




### フレームワークとは

フレームワークは、システム開発の世界でよく使われるツールの一種です。このフレームワークを使うと、**特定の種類のシステムをより速く、効率的に作ることができます。**

例えば、Web アプリケーションを作る時、画面を切り替えたり、ユーザのリクエストを受け取ったり、データを一時的に保持したりすることがよくあります。これらは、どんなWeb アプリケーションにも必要な基本的な機能です。

以前は、サーブレットという技術を使ってこれらの機能を実装していましたが、ちょっと複雑で時間がかかる作業でした。ですが、フレームワークを使うと、これらの作業を簡単なコードやキーワードで実装できるようになります。つまり、フレームワークは開発者の作業効率を高めてくれます。

図1. 1. 1

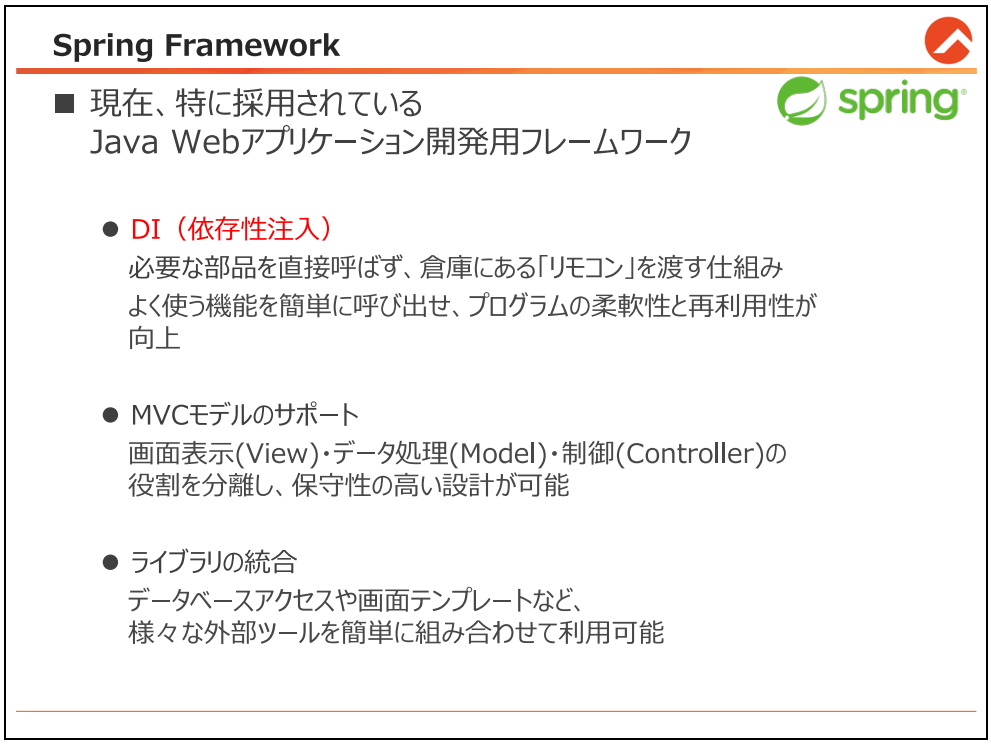



共通機能をフレームワークに任せ、開発者はアプリケーション固有の処理を組み立てます。

### フレームワークの注意点

フレームワークは便利ですが、フレームワークによって**定められたルールや方法に従って作業する必要**があります。最初は、これらのルールに慣れることに集中しましょう。また、エラーが発生した時の対処方法も、フレームワークを使いこなす上で大切です。エラーメッセージを見て、何が問題なのかを理解し、解決する力を身につけましょう。もし、問題が発生した場合は、本書付録の「よく起きるエラーまとめ」を参考にしてください。

## 1.2. Spring Framework




### Java Web アプリケーションの歴史とSpring Framework

「Spring Framework」、通称「Spring」は、Java 言語を使用してエンタープライズアプリケーション全体を開発するためのフレームワークです。Web アプリケーションの開発もその中の一部でした。

1995 年にJava が公開され、その後の1990 年代後半には、サーブレットとJSP を活用したWeb アプリケーション開発の時代が始まりました。サーブレット&JSP はJava Web アプリケーションの開発の基盤となりましたが、これらの技術だけでは複雑なアプリケーションの開発が難しく、冗長な作業が増えることが課題となりました。

2000 年代に入ると、この課題を解決するためのフレームワークがいくつか登場しました。Struts はApache Software Foundation から公開され、初期の頃の主要なMVC フレームワークとして広く採用されました。一方、2002 年にはSpring Framework がRod Johnson 氏によって初めて公開されました。Seasar2 などの他のフレームワークもこの時期に登場しました。

2010 年代に入ると、多くのJava フレームワークの中でもSpring はその柔軟性、拡張性、そして豊富なサブプロジェクト（例: Spring Boot, Spring Data）のおかげで非常に人気があり、多くの企業やプロジェクトで採用されるようになりました。その結果、多くのJava のフレームワークの中で、**Spring は特に注目される存在**となりました。

## Spring の特徴

### 1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。A クラスは渡されたリモコンの「スイッチ（メソッド）」を押すだけで機能を使えます。

図1. 2. 1




DIコンテナから渡されたリモコンを使い、AクラスがBクラスの機能を呼び出します。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

### 2. MVC モデルの設計理念

Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（View）、データの処理（Model）、そしてユーザの入力と画面表示の制御（Controller）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

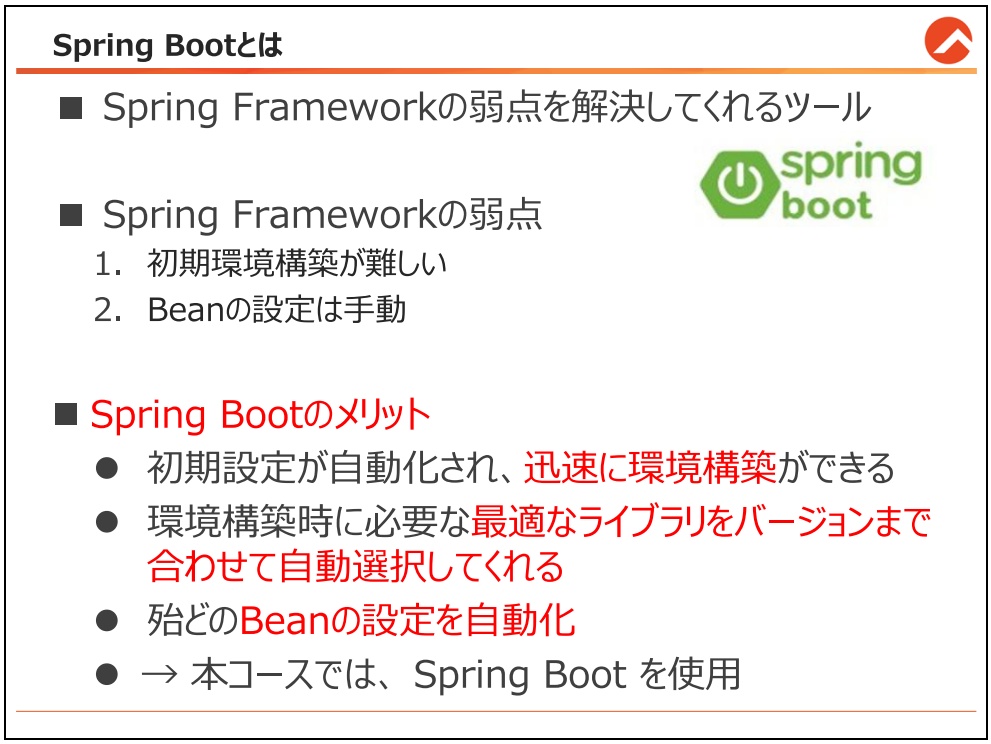
### 3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

**本書では、Spring Framework 7.0.3を利用します。**

## 1.3. Spring Boot とは




### Spring Boot とは

Spring Framework は機能が豊富なフレームワークですが、アプリケーションの開発を始める際には、**環境構築の手順が複雑で難しい**というデメリットがありました。

この課題を解決するために、Spring Boot が開発されました。当コースでは、Spring Boot を使用し、研修を進めます。Spring Boot は、Spring Framework をより簡単に使えるようにするツールのようなものです。

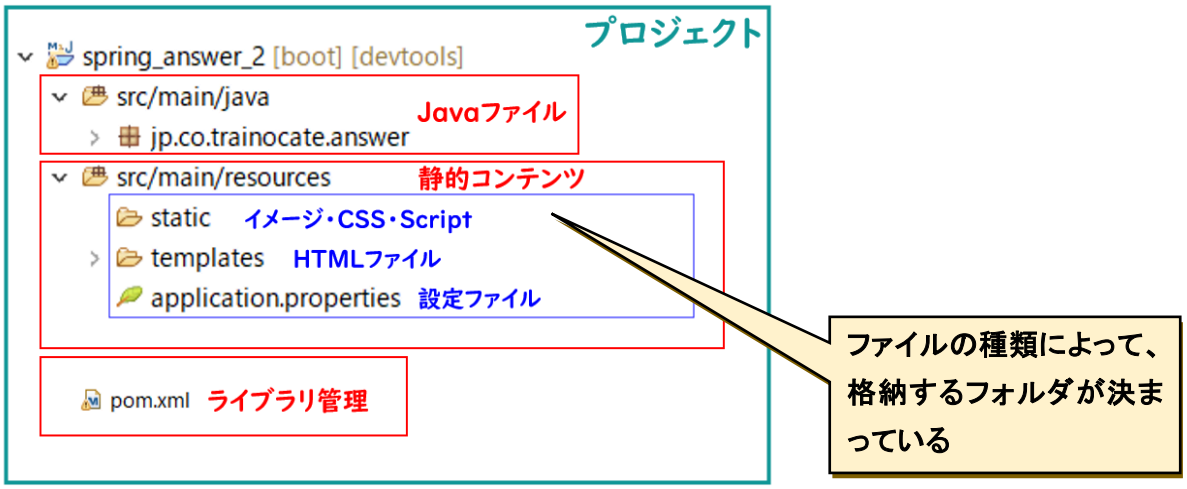
Spring Boot は、開発者が簡単に Web アプリケーションを作成できるように設計されています。以下にその主なメリットを紹介します。

### 1. セットアップが簡単

Spring Boot は、多くの設定を自動で行ってくれるため、開発を始めるための準備が簡単です。そのおかげで、プログラミングの初心者でも迅速にプロジェクトを立ち上げることができます。Spring Boot によって作成されたプロジェクトは、自動で以下のようなフォルダ構成になります。

**＜Spring Boot プロジェクトの構成＞**

図1. 3. 1




**Spring Boot では、ファイルの種類によって、決まったフォルダに格納しないと、正常動作しません。spring_answer_2 プロジェクトを展開して、ファイルの構成を確認してください。**

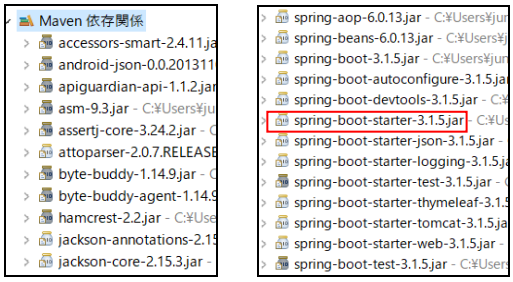
### 2. 依存ライブラリの簡単な管理

Spring Boot の次のメリットは、ライブラリのバージョン管理を自動で行ってくれることです。

通常、複数のライブラリを組み合わせて開発する場合、「どのライブラリの、どのバージョン同士なら競合せずに動くか」を調整するのは非常に手間がかかります。しかし Spring Boot では、**初期環境構築時にベースとなる「Spring Boot のバージョン」を決めるだけで、それに最適な関連ライブラリのバージョンを自動で選択してくれます。**

以下の画像は、Maven により必要なライブラリが自動導入されている様子です。開発者はバージョンの相性に悩まされることなく、スムーズに開発を始めることができます。

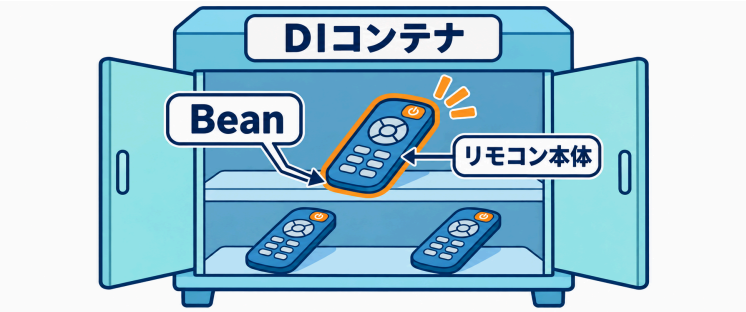
図1. 3. 2



### 3. Bean の自動設定

Spring Framework では、アプリケーションの多くの機能が**「Bean」**と呼ばれる特別なオブジェクトとして管理されます。Bean は、先ほどDI（依存性の注入）で例えた「予め倉庫（DIコンテナ）に準備されている、リモコン本体」のことです。

図1. 3. 3




Beanは、DIコンテナに準備されているリモコン本体に相当します。

通常、この Bean を倉庫（DI コンテナ）に登録・管理するためには、以下のように手動で一つ一つ設定を書く必要があります。

Java

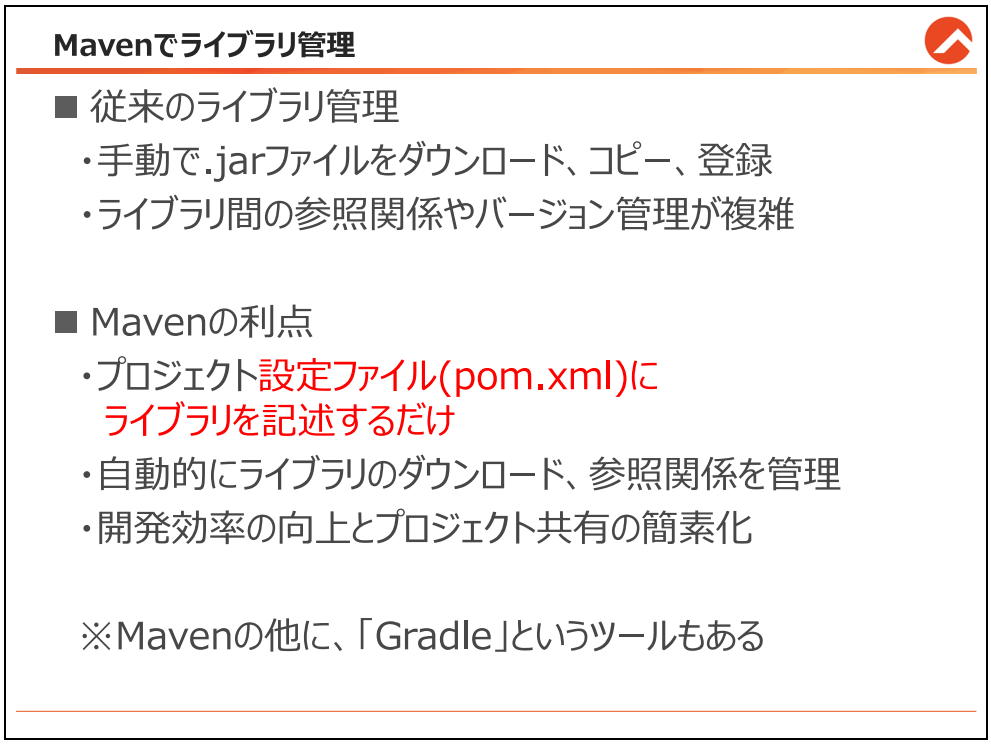
```java
@Configuration
public class AppConfig {
  //myService をBean に登録すると便利だが、登録に手間がかかる
        @Bean
        public MyServiceImpl myService() {
                return new MyServiceImpl();
        }
}
```

ただし、**Bean の数が多くなると、それぞれの設定が複雑になり、管理が難しくなります。Spring Boot は、多くの一般的なBean を、最初から自動でDI コンテナに準備・設定してくれます。**

開発者が手動で Bean を設定する手間が大幅に減るため、アプリケーションの本来の目的（ビジネスロジック）の実装により集中できるようになります。

このように、Spring Boot は Spring Framework の機能を拡張し、開発プロセスを簡略化して、より効率的にするための大変便利なツールとなっています。

## 1.4. Maven でライブラリ管理




### Maven でライブラリ管理

Spring Boot では、Maven などのビルドツールを使ってライブラリの管理をするのが一般的です。**Mavenを使うと、従来は複雑なライブラリの管理を簡単に行うことができる**ようになります。

Java Web プロジェクトを開発する際、データベースの接続やその他の多くの機能を実装するために、外部ライブラリの利用はほぼ必須です。しかし、従来の方法では以下のような手順が必要でした。

**【参考】従来のライブラリ導入方法**

1. インターネットから適切な.jar ファイル(例：mysql-connector-j.jar)を探し出し、ダウンロードする。

2. ダウンロードした.jar ファイルをプロジェクトの特定のフォルダにコピーする。

3. プロジェクトのビルドパスにこれらの.jar ファイルを登録する。

手動でのライブラリ管理は、時間がかかるだけでなく、ライブラリ間の参照関係やバージョンの整合性を手動で管理する必要があり、非常に面倒です。さらに、プロジェクトを他の開発者と共有する際も、全てのライブラリファイルを含める必要があるため、非効率です。

「Maven」や「Gradle」といったビルドツールを利用することで、問題を解決できます。これらのツールを使うと、**必要なライブラリをプロジェクトの設定ファイルに記述するだけで、ツールが自動的にライブラリをダウンロードし、参照関係を管理**してくれます。このおかげで、プロジェクトの設定と共有が簡単になり、開発効率が大幅に向上します。

### Maven を利用し、ライブラリを導入する手順

1. インターネットから、導入したいライブラリ名を検索し、[Maven Repository]に接続する。
   (今回の例では、MySQL Connector J と検索しました。)

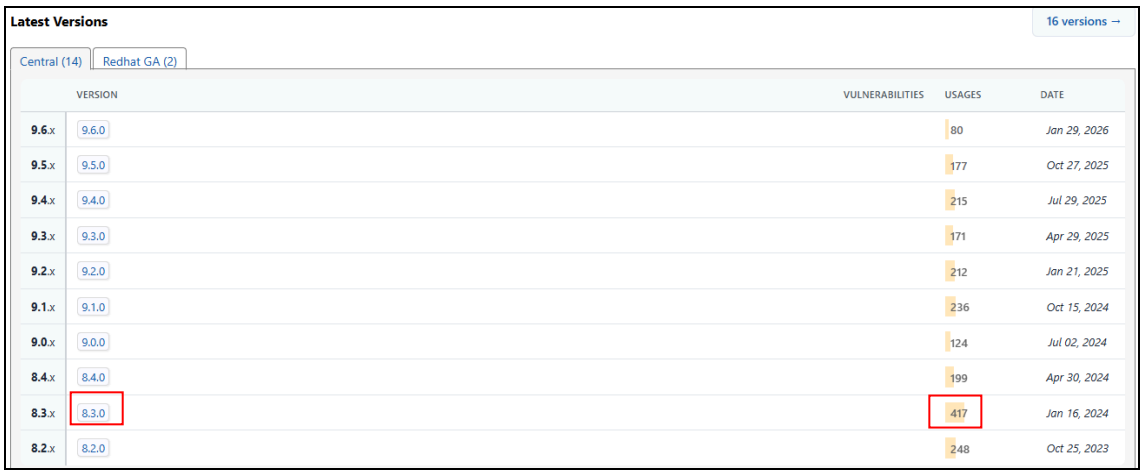
   (https://mvnrepository.com/artifact/com.mysql/mysql-connector-j)

図1. 4. 1



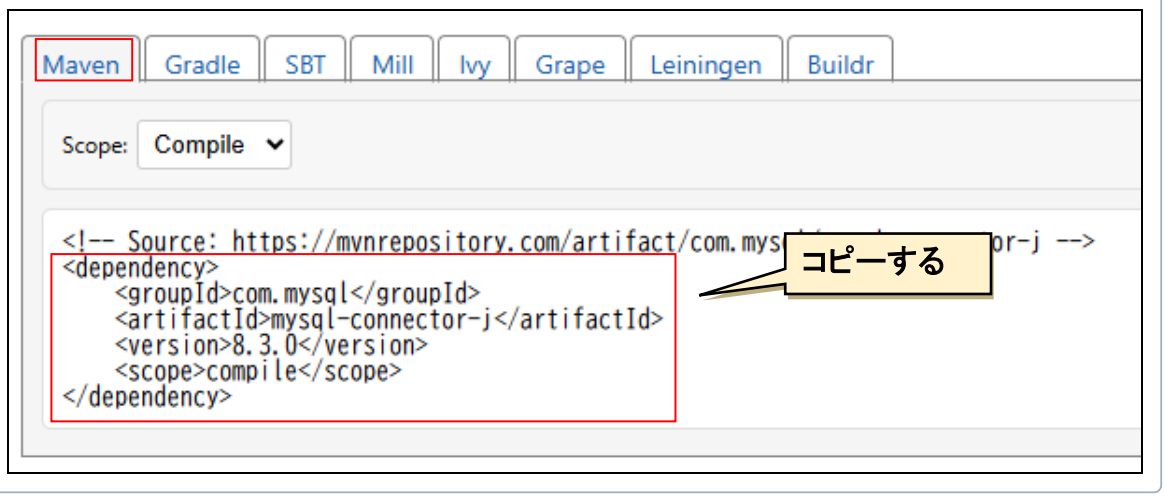

2. 導入したいバージョン名をクリックする。（よくわからない場合は、Usages が一番多い方を選ぶ）

図1. 4. 2



3. 利用するビルドツールに合わせて、タブを切り替えする。表示されたテキストをコピーしておく。

図1. 4. 3

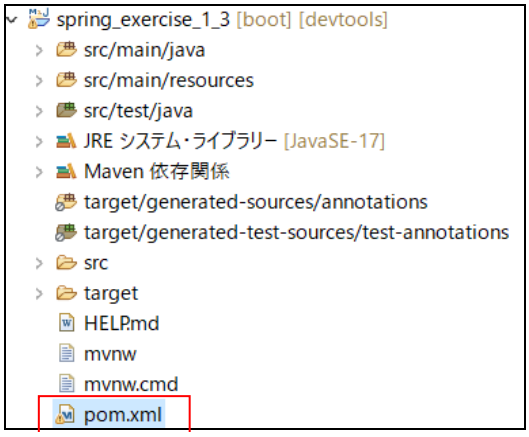



XML

```xml
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.3.0</version>
    <scope>compile</scope>
</dependency>
```

4. Eclipse 上の[spring_exercise_1_3]プロジェクトをダブルクリックし、[pom.xml]ファイルをダブルクリックする。

図1. 4. 4




5. **<dependencies>と、</dependencies>の間の任意の場所**に、手順 3 でコピーしたテキストを貼り付けて、保存する。

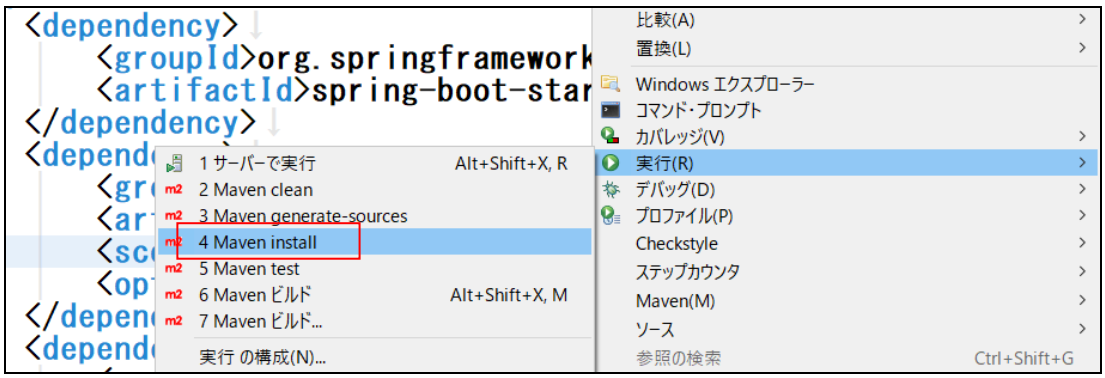
図1.4.5

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

        <!-- この場所に、コピーしたテキストを貼り付けしてください -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>8.3.0</version>
            <scope>compile</scope>
        </dependency>
    </dependencies>
```

6. pom.xml の任意の場所を右クリック→[実行]→[Maven install]をクリックすると、ライブラリの導入が完了する。

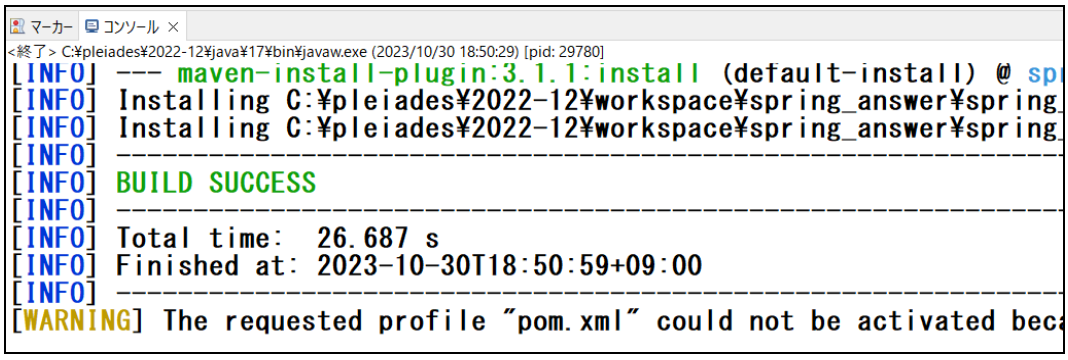
図1.4.6




7. コンソール上で、ダウンロードとインストールのログが流れる。
以下のように、[BUILD SUCCESS]が表示されると、完了。

※[FAILED]になった際は、誤字を確認して修正後、再度試す。

図1.4.7



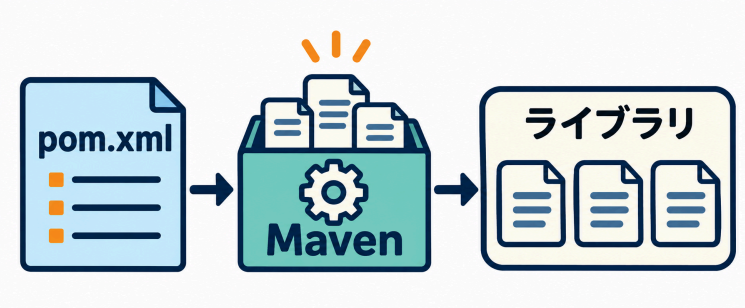
### Maven によるライブラリ管理の利点

以上が、Maven を利用したライブラリの適用方法です。Maven を使用する最大のメリットは、「プロジェクトを他の人と共有するのが非常に簡単になること」です。

通常、作成したプロジェクトを第三者に渡す際、使用している重いライブラリ（部品）のファイルを一つ一つセットにして渡す必要があります。しかし Maven を使えば、ライブラリの「お買い物リスト」である pom.xml ファイルだけを渡せば済みます。

プロジェクトを受け取った人が maven install を実行すると、リストに書かれた「全く同じバージョンのライブラリ」がインターネットから自動的にダウンロードされ、インストールされます。

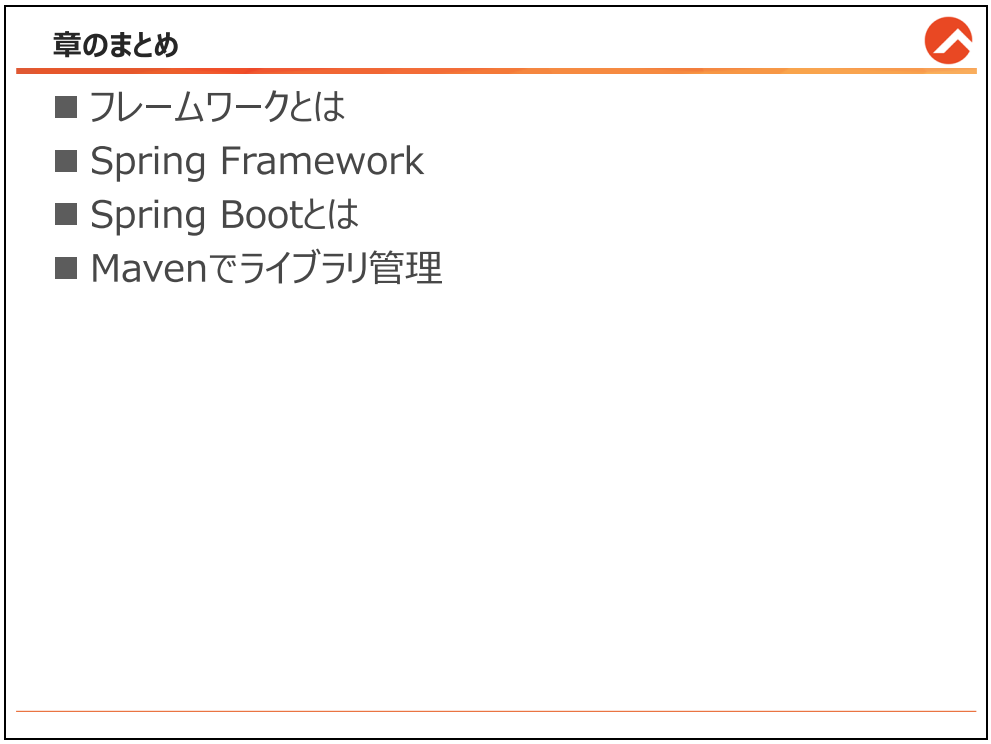
図1. 4. 8




pom.xmlの指定をもとに、Mavenが必要なライブラリをそろえます。

これにより、「A さんのパソコンでは動くのに、B さんのパソコンでは動かない」といった環境のズレを防ぎ、チーム全員が同じ環境で効率よく開発を進めることができます。

## 章のまとめ




この章では、フレームワークの概念とその必要性を理解しました。特に、Java の開発で広く利用されている Spring Framework の特徴と強みについて学びました。また、Spring Boot の導入によって、Spring Framework の開発をより手軽に始められる方法について説明しました。最後に、Mavenを使った効率的なライブラリ管理の方法を学び、プロジェクトの構築と管理がいかに簡単になるかを理解できました。

これで Spring Boot での Web アプリケーション開発への第一歩を踏み出す準備が整いましたので、次の章から本格的にSpring Boot を使ったWeb アプリケーションの開発に挑戦しましょう。